

Jarvis AI Investment Analysis MVP - High-Level Architecture

Version 0.1 (Phase 0 MVP) | 2026-01-10

This document describes the end-to-end technical architecture, orchestration model, and data lifecycle for the Jarvis MVP: a portfolio-driven system that ingests financial market data and patent intelligence, then stores AI-ready artefacts for downstream insight generation.

1. System boundary and goal

A user creates a portfolio (tickers + thesis) in the front-end. The backend orchestrates per-ticker ingestion for financial and patent data, then performs portfolio-level analytics once all tickers complete. Results are persisted in Supabase for retrieval by the UI.

2. Core components

Component	Role in MVP
Frontend (Lovable)	Collect portfolio inputs, show status/progress, display outputs and downloadable artefacts.
Supabase (Postgres)	System of record for portfolios, track_requests, ticker_data, patent_data. Drives event triggers.
Supabase Database Webhooks	Emit events (INSERT/UPDATE) to n8n when rows change.
n8n	Workflow engine: fan-out, enrichment, ingestion, status updates, and completion checks.
Jarvis Market Data Service (FastAPI)	Fetch OHLCV + fundamentals; resolve ticker → legal company name.
PatentsView API	Fetch patents by legal assignee name (company_name).
LLM (OpenAI)	Summarise patents and generate insights; store in JSONB payloads.

3. High-level diagram (logical view)

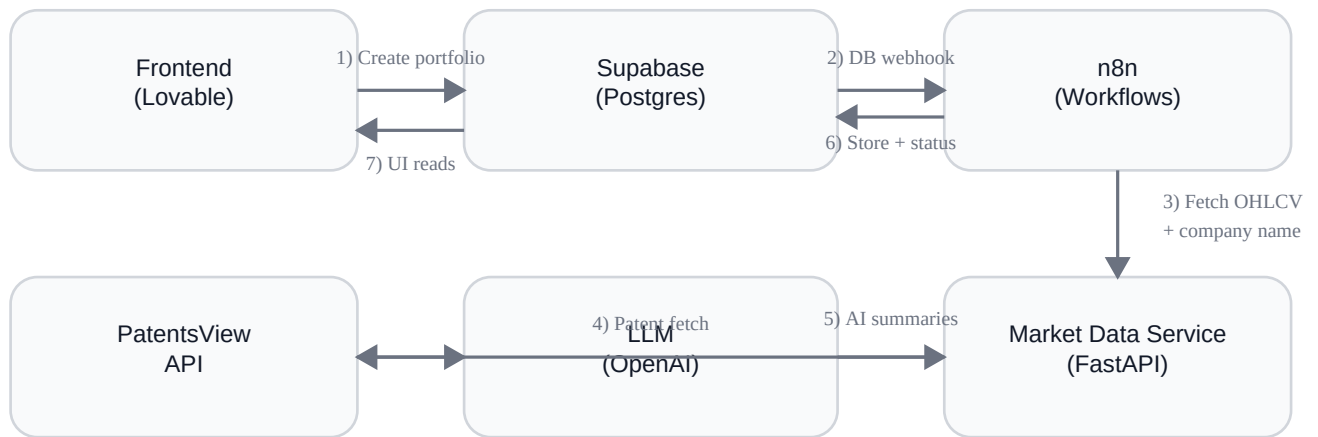


Diagram note: This is a logical view. In practice, n8n mediates calls to FastAPI, PatentsView, and the LLM, and persists all artefacts and statuses back into Supabase.

4. Data model and status tracking

Jarvis uses an event-driven fan-out/fan-in pattern. A **portfolio** is the user-level object. Each portfolio expands into **track_requests** rows (one per ticker), each with independent completion states for financial and patent ingestion. Portfolio-level status fields summarise progress across child requests.

Key identifiers:

- **portfolio_id**: groups all tickers for one user portfolio.
- **request_id**: per-ticker unique id used as the correlation key across workflows; reused as `batch_id` when calling `/portfolio/json`.

5. Workflow-level data cycle (Phase 0 MVP)

Workflow 1 - Dispatcher (Portfolio → Requests)

Trigger: INSERT on portfolios. n8n reads `input_tickers`, creates one `track_requests` row per ticker, resolves `company_name` via `/company/name`, and sets initial portfolio status (e.g., `fdi_status='pending'`, `pdi_status='pending'`).

Workflow 2 - FDIgestion (Financial Data Ingestion)

Trigger: INSERT on `track_requests`. For each `request_id`: call `POST /portfolio/json` (single ticker), write `raw_ohlc` into `ticker_data`, update `track_requests.financial_analysis_status='complete'`, then trigger patent ingestion.

Workflow 3 - PDIgestion (Patent Data Ingestion)

Trigger: webhook from FDIgestion (or direct DB trigger). For each `request_id`: query PatentsView using `company_name`, run LLM summarisation, store `summary_payload` into `patent_data`, update `track_requests.patent_analysis_status='complete'`.

6. Synchronisation: portfolio-level completion gate

To avoid duplicate portfolio updates while still creating N track_requests for N tickers, use a completion gate: count child rows that reached 'complete' and update the portfolio once when the count equals ticker_count.

End of document.